Unit 5: Introduction to Programming

Level: **2**

Unit type: **Internal**

Guided learning hours: **60**

Unit in brief

Learners study the key features of programming languages to develop and test software solutions.

Unit introduction

Being able to create software solutions could give you a brilliant opportunity for a varied, interesting, and challenging career that few other vocational sectors can match. Software developers or engineers develop, test, and maintain computer programs that can solve problems or improve productivity.

To create successful software solutions, you must develop good problem-solving and creative-thinking skills.

In this unit, you will learn about different programming paradigms and how to write code to develop a successful program in a chosen language.

You will develop your own software solution to fulfil a brief. You will test your solution for functionality and identify and repair faults. You will review your finished program, and evaluate possible improvements.

This unit is not platform dependent, and a range of program-development environments can be used.

Learning aims

In this unit you will:

A Understand the key features of different programming paradigms

B Develop and test a software solution

C Review the software solution.

Summary of unit

Learning aim	Key content areas	Assessment approach
A Understand the key features of different programming paradigms	**A1** Features of event- driven programming **A2** Features of object-oriented programming **A3** Features of procedural programming	An investigation into each programming paradigm and their suitability for a particular problem(s).
B Develop and test a software solution	**B1** Developing software **B2** Testing and refining a software solution	A working program in one programming language. Testing documentation that includes the outcomes of testing and any refinements made during development. Refine the solution based on testing. A review on the suitability of the software solution.
C Review the software solution	**C1** Software solution review	

Content

Learning aim A: Understand the key features of different programming paradigms

A1 Features of event-driven programming

Features of event-driven programming paradigm, and the comparative advantages and disadvantages to other approaches.

- Features:
  - event loop
  - event trigger
  - event handlers and listeners
  - top-down approach.
- Advantages, such as:
  - simple to learn
  - easy development, e.g. additional functionality through programming events such as control objects
  - flexibility, e.g. code follows a logical order from start to finish.
- Disadvantages, such as:
  - impact on system resources
  - translation limitations
  - program flow.

A2 Features of object-oriented programming

Features of object-oriented programming paradigm, and the comparative advantages and disadvantages to other approaches.

- Features:
  - identification of objects
  - data abstraction
  - modularity
  - classification
  - inheritance
  - polymorphism
  - encapsulation
  - classes

    - methods
    - bottom-up approach.
- Advantages, such as:
    - modular structure
    - easier troubleshooting
    - less coding required when developing larger programs.
- Disadvantages, such as:
    - larger program
    - more complex coding
    - a higher skill level is required than other paradigms.

A3 Features of procedural programming

Features of procedural programming paradigms, and the comparative advantages and disadvantages to other approaches.

- Key features:
  - procedures
  - functions
  - pre-defined functions
  - local variables
  - global variables
  - top-down approach.
- Advantages, such as:
  - easier to code than object-oriented
  - less memory required than for event-driven paradigm.
- Disadvantages, such as:
  - difficult to expand
  - hard to maintain.

Learning aim B: Develop and test a software solution

B1 Developing software

Processes and operations required to develop and refine software.

- Constructs and techniques
  - constants (variables with a constant value that cannot change)
  - operators (arithmetic [+, –, *, /, %] and logical [<, <=, >, >=, AND, OR, true, false])
  - reserved words
  - input and output commands
  - local variables
  - global variables
  - objects/classes/methods
  - sequence
  - selection
  - iteration

- subroutines/functions/procedures
- data types:
  - character
  - string (text)
  - integer and real (numbers)
  - boolean.
- use of data structures:
  - user-defined data types and record structures
  - arrays.
- event handling:
  - forms
  - assigning properties to screen components, e.g. buttons, boxes, data validation and drop-down lists
  - actions.

- Good coding practices:
  - comments
  - indentation
  - suitable variable names.

B2 Testing and refining a software solution

Processes required to test and refine a software solution.

- Establish functionality against a test plan with the test data (normal, abnormal and extreme).
- Quality of code, e.g. maintainability, portability and usability.
- Document any improvements/refinements that are made to the software solution.

Learning aim C: Review the software solution

C1 Software solution review

- Criteria for reviewing effectiveness of software solution:
  - user requirements
  - fitness for purpose
  - user experience, e.g. ease of navigation
  - constraints, e.g. programming language, time, device capabilities, memory, connectivity
  - quality of the program, e.g. reliability, usability, efficiency/performance, maintainability, portability
  - strengths and improvements.

Assessment criteria

Pass	Merit	Distinction
Learning aim A: Understand the key features of different programming paradigms		
A.P1 Outline the key features of event-driven, object-oriented and procedural programming paradigms. **A.P2** Outline the reasons each paradigm is suitable in given situations.	**A.M1** Explain the features and suitability of event-driven, object-oriented and procedural programming paradigms for different software development purposes.	**A.D1** Evaluate the suitability of event-driven, object-oriented and procedural programming paradigms for a proposed software solution.
Learning aim B: Develop and test a software solution		
B.P3 Develop a program solution for a given brief. **B.P4** Test and refine the software solution for functionality and quality.	**B.M2** Refine and improve the developed program against the test results.	**BC.D2** Refine the developed program against the test results, justifying changes made to the quality of the code to make it fit for purpose, and making recommendations for further improvement.
Learning aim C: Review the software solution		
C.P5 Identify reasons for changes made at the testing stage. **C.P6** Describe how the finished software solution meets the user requirements and is fit for purpose.	**C.M3** Explain reasons for changes made at the testing stage. **C.M4** Explain how the finished software solution meets the user requirements and is fit for purpose.	

Essential information for assignments

The recommended structure of assessment is shown in the unit summary, along with suitable forms of evidence. *Section 6 Internal assessment* gives information on setting assignments and there is also further information on our website.

There is a suggested maximum number of two summative assignments for this unit. The relationship of the learning aims and criteria is:

Learning aim: A (A.P1, A.P2, A.M1, A.D1)

Learning aims: B and C (B.P3, B.P4, C.P5, C.P6, B.M2, C.M3, C.M4, BC.D2)

Further information for teachers and assessors

Resource requirements

For learners to have every opportunity to achieve success in this unit, they will require full access to at least one IDE suitable for writing, editing and testing code in the program language selected by the centre.

Essential information for assessment decisions Learning aim A

For Distinction standard, learners will provide a clear and balanced evaluation of

how suitable the three programming paradigms are for different proposed solutions. Three different programming paradigms from the unit content must be used and the advantages and disadvantages of applying them to different proposed solutions evaluated.

The evidence will demonstrate high-quality written/oral communication through use of accurate and fluent technical vocabulary to support a well-structured and considered response that clearly connects chains of reasoning. It will demonstrate a very good understanding of each programming paradigm with no inaccuracies or omissions.

For Merit standard, learners will explain the suitability of the three programming paradigms are for different proposed solutions. They will explain the different programming paradigms as outlined in the unit content.

Overall, the evidence will be logically structured. It will demonstrate a sound understanding of each programming paradigm. The evidence may contain minor inaccuracies or omissions.

For Pass standard, learners will outline the key features of each programming paradigm from the unit content along with the reasons why they could be suitable for applying to different proposed solutions.

Overall, the evidence will be basic in parts and may contain some inaccuracies or omissions.

Learning aims B and C

For Distinction standard, learners must draw on knowledge across the learning aims to optimise the software solution, taking account of test results and the quality of code,

e.g. maintainability, portability and usability.

They should justify these changes in terms of the requirements and the features of the language used, and any other constraints. Learners should make at least three specific suggestions for improving the completed program to ensure it is fully functional, well coded and fit for purpose. Learners do not need to implement the enhancements.

For Merit standard, learners must make use of the test results to refine the software solution.

Learners must explain the changes made during testing, and how the solution meets the requirements. The evidence will demonstrate a sound understanding of testing and refining processes. The explanation will be appropriate, but may have minor inaccuracies or omissions.

For Pass standard, learners must use an appropriate programming language to develop a solution for a given brief.

Learners must show that they have tested the solution to make sure it meets identified requirements, refining the solution as required. Learners must demonstrate how the solution meets the requirements, is fit for purpose, and identify any changes made during testing.

Learner descriptions of the effectiveness of the solution will be generally accurate. Their evidence may be basic in parts, for example covering more generic statements that do not link to the context. The evidence may contain some inaccuracies or omissions.

Links to other units and curriculum subjects

This unit links to:

- Unit 8: Introduction to App Development
- International GCSE/core curriculum in Information Technology.

Opportunities to develop transferable employability skills

In completing this unit, learners will have the opportunity to develop skills in research and planning, problem-solving, responsibility, and communication.